

# claude-windows / 42211f80-2169-4025-ad9d-99c1aa29ff70

## Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\42211f80-2169-4025-ad9d-99c1aa29ff70\subagents\workflows\wf_6237b110-d58\agent-ad46001e3390ccfee.jsonl`  
- SHA-256: `4023561340b107f2ac51c0e73a6ffc57cb0ed36eb268fe72b2feb8c79c00b738`  
- Source modified: `2026-06-06T17:38:57+00:00`  
- Imported at: `2026-07-05T16:48:28+00:00`  
- project: `wf_6237b110-d58`  
- session_id: `42211f80-2169-4025-ad9d-99c1aa29ff70`
```

## Transcript

### 1. user (2026-06-06T17:37:40.368Z)

#### Adversarial Claim Verifier (voter 1/3)

Be SKEPTICAL. Try to REFUTE this claim. ?2/3 refutations kill it.

#### Research question

Research alternative methods to automatically detect rally start/end segments (temporal segmentation) in racket-sport video ? especially badminton ? for a SELF-HOSTED, LOCAL-FIRST pipeline, and assess whether a cloud-VLM teacher ? local-model distillation is the best approach or whether other methods are competitive or complementary. Keep it focused ("mini" research): prioritize actionable, commercially-licensable, locally-runnable approaches over exotic SOTA needing huge compute or restrictive licenses.

OUR CONTEXT / CONSTRAINTS (ground every recommendation in these):

- Self-hosted on a single consumer GPU (RTX 2070 Super), Windows + WSL2. We want LOCAL-ONLY inference at runtime ? privacy lever: user video never leaves the device. It's a commercial product, so PERMISSIVE / commercial-friendly licenses are REQUIRED (avoid AGPL and non-commercial-only weights/datasets).
- Current substrate: WASB (MIT) shuttle-trajectory detector ? per-frame (x, y, visibility). We derive rally windows via velocity/merge-gap/min-duration thresholds plus a net-crossing gate. We ALSO have a permissive person detector available (torchvision Faster R-CNN + supervision ByteTrack) and the video's AUDIO track.
- Current best approach: use paid Gemini (cloud VLM, full-context) OFFLINE as a TEACHER to label rallies, then DISTILL into a local model (logistic regression on shuttle-trajectory features). Findings so far: threshold-tuning ceilings around F1 ? 0.44 and overfits with few videos; the learned model is data-starved at n=2 labeled videos; WASB candidate RECALL is a real bottleneck (its windows sometimes don't align with true rallies at IoU 0.5).
- Inputs we can exploit locally: video frames, the shuttle trajectory, player bounding boxes / pose (if we add a permissive pose model), and the AUDIO track (shuttle "thwack" hits, crowd noise, umpire/score calls).

SPECIFICALLY COVER:

1. Method families for rally/point segmentation in racket sports (badminton, tennis, table tennis, pickleball): audio-based shot/hit detection; temporal action localization (TAL) deep models; pose/player-stance + court-homography state machines (e.g., "serve-ready stance" gating); classical change-point detection / HMM on the shuttle trajectory; scoreboard / score-overlay OCR for point boundaries; self-supervised / weakly-supervised approaches; and VLM teacher?student distillation.
2. For EACH family: how it works, the labels/data it needs, expected accuracy, local-inference COMPUTE cost, and LICENSE / commercial viability of any models, datasets, or weights involved.
3. Badminton rally-segmentation datasets (e.g., ShuttleSet, TrackNet/TrackNetV2 data) and their licenses / commercial usability.
4. A RANKED recommendation for our local-first, permissive, single-GPU setup: is Gemini-teacher ? local-distillation the best, or are some alternatives competitive or complementary ?

especially AUDIO-based hit detection and pose/court-geometry state machines, which we can run fully locally?

5. Concrete, cheap next experiments we could run with what we already have (shuttle trajectory + person detector + audio).

### Claim under review

"The paper presents a deep-learning pipeline that includes rally-wise video trimming (a form of temporal rally segmentation) using shot-angle recognition, achieving 99% accuracy on shot angle recognition for that trimming step."

**\*\*Source:\*\*** <https://arxiv.org/pdf/2307.16000> (primary)

**\*\*Supporting quote:\*\*** "we achieved 99% accuracy on shot angle recognition for video trimming, over 92% accuracy for applying player keypoints sequences on shuttlecock flying direction prediction, and reported the evaluation results of rally-wise video trimming and hit-frame detection."

### Checklist

1. Is the claim actually supported by the quote, or is it an overreach/misread?
2. WebSearch for contradicting evidence ? does any credible source dispute or heavily qualify this?
3. Is the source quality sufficient for the claim's strength? (extraordinary claims need primary sources)
4. Is the claim outdated? (check dates ? old claims about fast-moving fields are suspect)
5. Is this a marketing claim / press release / cherry-picked benchmark / forum speculation?

**\*\*refuted=true\*\*** if: unsupported by quote / contradicted / low-quality source for strong claim / outdated / marketing fluff.

**\*\*refuted=false\*\*** ONLY if: claim is well-supported, current, and source quality matches claim strength.

Default to refuted=true if uncertain.

Structured output only. Evidence MUST be specific.

### 2. assistant (2026-06-06T17:37:43.012Z)

I'll verify this claim about the paper at [arxiv.org/pdf/2307.16000](https://arxiv.org/pdf/2307.16000). Let me load the web tools and fetch the source.

### 3. assistant (2026-06-06T17:37:46.260Z)

I'll fetch the actual paper to verify the claim about rally-wise video trimming via shot-angle recognition.

### 4. user (2026-06-06T17:37:52.929Z)

I cannot extract the requested information from the provided content. The PDF stream is binary-encoded and compressed (FlateDecode format), making it unreadable without proper decompression tools.

To answer your questions about:

1. Rally-wise video trimming and temporal rally segmentation
2. Shot-angle recognition usage
3. Shot angle recognition accuracy (99%?)
4. Definition of "shot angle"

I would need the document in readable text format. Please provide the paper as:

- A text file
- An uncompressed PDF
- A direct excerpt of the relevant sections

Alternatively, if you have access to the original paper, sharing its title and authors would allow me to locate it through academic databases.

[Binary content (application/pdf, 1.5MB) also saved to C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\42211f80-2169-4025-ad9d-99c1aa29ff70\tool-results\webfetch-1780766977975-dnjzkl.pdf]

#### 5. assistant (2026-06-06T17:37:55.476Z)

The PDF was downloaded but couldn't be parsed by the fetch tool. Let me read it directly and also search for the abstract page.

#### 6. user (2026-06-06T17:37:58.058Z)

pdftoppm failed: Command 'pdftoppm' not found or is in an unsafe location (current directory)

#### 7. user (2026-06-06T17:38:03.215Z)

## Paper Summary

**Title:** Automated Hit-frame Detection for Badminton Match Analysis

**Authors:** Yu-Hang Chien and Fang Yu

**Date:** Submitted July 29, 2023; revised August 2, 2023

**About the Paper:**

This research develops automated methods to identify hit-frames in badminton match videos using deep learning. The system enables coaches and players to analyze performance systematically without manual effort, supporting downstream applications like stroke synthesis and strategy evaluation.

**Mentioned Concepts:**

Yes, the paper explicitly references all four elements you asked about. The abstract states: "We achieved 99% accuracy on shot angle recognition for video trimming, over 92% accuracy for applying player keypoints sequences on shuttlecock flying direction prediction, and reported the evaluation results of rally-wise video trimming and hit-frame detection."

**Abstract (Verbatim):**

"Sports professionals constantly under pressure to perform at the highest level can benefit from sports analysis, which allows coaches and players to reduce manual efforts and systematically evaluate their performance using automated tools. This research aims to advance sports analysis in badminton, systematically detecting hit-frames automatically from match videos using modern deep learning techniques. The data included in hit-frames can subsequently be utilized to synthesize players' strokes and on-court movement, as well as for other downstream applications such as analyzing training tasks and competition strategy. The proposed approach in this study comprises several automated procedures like rally-wise video trimming, player and court keypoints detection, shuttlecock flying direction prediction, and hit-frame detection. In the study, we achieved 99% accuracy on shot angle recognition for video trimming, over 92% accuracy for applying player keypoints sequences on shuttlecock flying direction prediction, and reported the evaluation results of rally-wise video trimming and hit-frame detection."

#### 8. assistant (2026-06-06T17:38:07.270Z)

The abstract is now confirmed verbatim. Let me read the PDF body to understand what "shot angle recognition" actually means in this paper and how it relates to "rally-wise video trimming," since these are described as two distinct procedures in the abstract.

#### 9. user (2026-06-06T17:38:08.765Z)

pypdf available

#### 10. user (2026-06-06T17:38:16.667Z)

PAGES: 8

==== Kw: shot angle ====

ing training tasks and competition strategy. The proposed approach in this study comprises several automated procedures like rally-wise video trimming, player and court keypoints detection, shuttlecock flying direction prediction, and hit-frame detection. In the study, we achieved 99% accuracy on shot angle recognition for video trimming, over 92% accuracy for applying player keypoints sequences on shuttlecock flying direction prediction, and reported the evaluation results of rally-wise video trimming and hit-frame detection. Introduction Badminton, the fastest racquet sport in the world, is played

==== KW: shot angle ====

st the shuttlecock's direction of flight in each frame, and pinpoint hit-frames. Algorithm 1 depicts the process of all the procedures. The hit-frame generating algorithm inputs a badminton frame sequence (video)  $V$  and produces a set of hit-frame indices denoted by  $Hit$ . Lines 1 to 3 define the shot angle constants  $O$  (representing other) and  $H$  (representing high). These constants are used to realize the rally-wise video trimming module (described in ). The variable previous shot angle is initialized to  $O$  to keep track of the shot angle of the previous frame. Two sets are also created:  $Kps$  (for s

==== KW: shot angle ====

badminton frame sequence (video)  $V$  and produces a set of hit-frame indices denoted by  $Hit$ . Lines 1 to 3 define the shot angle constants  $O$  (representing other) and  $H$  (representing high). These constants are used to realize the rally-wise video trimming module (described in ). The variable previous shot angle is initialized to  $O$  to keep track of the shot angle of the previous frame. Two sets are also created:  $Kps$  (for storing detected player keypoints) and  $Hit$  (for storing detected hit-frame indices), both initially empty. From line 4, the algorithm proceeds to iterate through each frame  $f$  in the

==== KW: shot angle ====

of hit-frame indices denoted by  $Hit$ . Lines 1 to 3 define the shot angle constants  $O$  (representing other) and  $H$  (representing high). These constants are used to realize the rally-wise video trimming module (described in ). The variable previous shot angle is initialized to  $O$  to keep track of the shot angle of the previous frame. Two sets are also created:  $Kps$  (for storing detected player keypoints) and  $Hit$  (for storing detected hit-frame indices), both initially empty. From line 4, the algorithm proceeds to iterate through each frame  $f$  in the input  $V$ . For each frame, the shot angle is recog

==== KW: shot angle ====

ame, the shot angle is recognized using the function  $AngleRecognition$  (described in ), and the algorithm follows a series of conditional checks to identify consecutive frames belonging to the same rally. 1. If the angle of the current frame matches the angle of the previous frame previous shot angle and equals to  $H$  (like line 6), which indicates the rally is still ongoing, the algorithm detects the keypoints  $kps$  of such frames using  $DetectPlayerKeypoints$  (elaborated in ) and add  $kps$  to the  $Kps$  set. 2. If angle matches previous shot angle and equals  $O$  (like line 10), the algorithm will overlo

==== KW: shot angle ====

angle matches the angle of the previous frame previous shot angle and equals to  $H$  (like line 6), which indicates the rally is still ongoing, the algorithm detects the keypoints  $kps$  of such frames using  $DetectPlayerKeypoints$  (elaborated in ) and add  $kps$  to the  $Kps$  set. 2. If angle matches previous shot angle and equals  $O$  (like line 10), the algorithm will overlook such frame and iterate back to line 4. Algorithm 1: Hit-frame generating algorithm. Input: Badminton frame sequence  $V$ . Output: Hit-frame set  $Hit$ . 1: Let const  $O = 0, H = 1$  2: Let previous shot angle  $= O$  3: Let  $Kps = \emptyset, Hit = \emptyset$  4: for all  $f$  in  $V$  do

==== KW: shot angle ====

o the  $Kps$  set. 2. If angle matches previous shot angle and equals  $O$  (like line 10), the algorithm will overlook such frame and iterate back to line 4. Algorithm 1: Hit-frame generating algorithm. Input: Badminton frame sequence  $V$ . Output: Hit-frame set  $Hit$ . 1: Let const  $O = 0, H = 1$  2: Let previous shot angle  $= O$  3: Let  $Kps = \emptyset, Hit = \emptyset$  4: for all  $f$  in  $V$  do 5: angle  $= AngleRecognition(f)$  6: if angle  $=$  previous shot angle then 7: if shot angle  $= H$  then 8:  $kps = DetectPlayerKeypoints(f)$  9:  $Kps.add(kps)$  10: else 11: continue 12: end if 13: else 14: if shot angle  $= O$  then 15: s  $= ShuttlecockDirP$

==== KW: shot angle ====

e and iterate back to line 4. Algorithm 1: Hit-frame generating algorithm. Input: Badminton frame sequence  $V$ . Output: Hit-frame set  $Hit$ . 1: Let const  $O = 0, H = 1$  2: Let previous shot

```

angle ? 0 3: Let Kps ? ?, Hit? ? 4: for all f rame? V do 5: angle ? AngleRecognition(f rame) 6:
if angle = previous shot angle then 7: if shot angle = H then 8: kps ? DetectPlayerKeypoints(f
rame) 9: Kps.add(kps) 10: else 11: continue 12: end if 13: else 14: if shot angle = 0 then 15: s
? ShuttlecockDirPrediction(Kps) 16: hit ? HitFrameDetection(s) 17: Hit.add(hit) 18: Kps ? ? 19:
else 20: kps ? DetectPlayerKeypoints(f ram

```

==== KW: shot angle ====

line 4. Algorithm 1: Hit-frame generating algorithm. Input: Badminton frame sequence V . Output: Hit-frame set Hit. 1: Let const 0 ? 0, H? 1 2: Let previous shot angle ? 0 3: Let Kps ? ?, Hit? ? 4: for all f rame? V do 5: angle ? AngleRecognition(f rame) 6: if angle = previous shot angle then 7: if shot angle = H then 8: kps ? DetectPlayerKeypoints(f rame) 9: Kps.add(kps) 10: else 11: continue 12: end if 13: else 14: if shot angle = 0 then 15: s ? ShuttlecockDirPrediction(Kps) 16: hit ? HitFrameDetection(s) 17: Hit.add(hit) 18: Kps ? ? 19: else 20: kps ? DetectPlayerKeypoints(f rame) 21: Kps.add(kps) 22: end if 23: end if 24: previous shot angle ? shot angle 25: end for 26: return Hit 3. If angle differs from previous shot a

==== KW: shot angle ====

onst 0 ? 0, H? 1 2: Let previous shot angle ? 0 3: Let Kps ? ?, Hit? ? 4: for all f rame? V do 5: angle ? AngleRecognition(f rame) 6: if angle = previous shot angle then 7: if shot angle = H then 8: kps ? DetectPlayerKeypoints(f rame) 9: Kps.add(kps) 10: else 11: continue 12: end if 13: else 14: if shot angle = 0 then 15: s ? ShuttlecockDirPrediction(Kps) 16: hit ? HitFrameDetection(s) 17: Hit.add(hit) 18: Kps ? ? 19: else 20: kps ? DetectPlayerKeypoints(f rame) 21: Kps.add(kps) 22: end if 23: end if 24: previous shot angle ? shot angle 25: end for 26: return Hit 3. If angle differs from previous shot a

==== KW: shot angle ====

ypoints(f rame) 9: Kps.add(kps) 10: else 11: continue 12: end if 13: else 14: if shot angle = 0 then 15: s ? ShuttlecockDirPrediction(Kps) 16: hit ? HitFrameDetection(s) 17: Hit.add(hit) 18: Kps ? ? 19: else 20: kps ? DetectPlayerKeypoints(f rame) 21: Kps.add(kps) 22: end if 23: end if 24: previous shot angle ? shot angle 25: end for 26: return Hit 3. If angle differs from previous shot angle and equals 0 (like line 14), it indicates the end of a rally, the algorithm predicts the flying directions of the shuttlecock using keypoints stored in Kps (elaborated in ) and assigns it to s. Then, it proceed

==== KW: shot angle ====

e) 9: Kps.add(kps) 10: else 11: continue 12: end if 13: else 14: if shot angle = 0 then 15: s ? ShuttlecockDirPrediction(Kps) 16: hit ? HitFrameDetection(s) 17: Hit.add(hit) 18: Kps ? ? 19: else 20: kps ? DetectPlayerKeypoints(f rame) 21: Kps.add(kps) 22: end if 23: end if 24: previous shot angle ? shot angle 25: end for 26: return Hit 3. If angle differs from previous shot angle and equals 0 (like line 14), it indicates the end of a rally, the algorithm predicts the flying directions of the shuttlecock using keypoints stored in Kps (elaborated in ) and assigns it to s. Then, it proceeds to detect h

==== KW: shot angle ====

angle = 0 then 15: s ? ShuttlecockDirPrediction(Kps) 16: hit ? HitFrameDetection(s) 17: Hit.add(hit) 18: Kps ? ? 19: else 20: kps ? DetectPlayerKeypoints(f rame) 21: Kps.add(kps) 22: end if 23: end if 24: previous shot angle ? shot angle 25: end for 26: return Hit 3. If angle differs from previous shot angle and equals 0 (like line 14), it indicates the end of a rally, the algorithm predicts the flying directions of the shuttlecock using keypoints stored in Kps (elaborated in ) and assigns it to s. Then, it proceeds to detect hit-frames hit using HitFrameDetection (illustrated in ) and adds it to t

==== KW: shot angle ====

s of the shuttlecock using keypoints stored in Kps (elaborated in ) and assigns it to s. Then, it proceeds to detect hit-frames hit using HitFrameDetection (illustrated in ) and adds it to theHit set. The Kps set is then cleared, waiting for a new rally to begin. 4. If angle differs from previous shot angle and equals H (line 19), it indicates the start of a rally. The algorithm will process such frames like those in lines 8 and 9. After the algorithm processes all frames in the input sequence, it returns the set Hit, which contains detected hit-frames indices of the input video. Contribution The main contributions of our article are to: 1.

==== KW: shot angle ====

hm will process such frames like those in lines 8 and 9. After the algorithm processes all frames in the input sequence, it returns the set Hit, which contains detected hit-frames indices of the input video. Contribution The main contributions of our article are to: 1. Utilize the interchange of shot angle to trim rally-wise frames from raw badminton video. 2.

Proposed a novel transformer that predicts shuttlecock direction sequences based on player keypoint sequence. 3. Designed and developed the first-ever automated hit-frame detection tool to bridge the gap between raw badminton videos and analysis

==== KW: shot angle ====

and labeled each frame as either high (class 1) or other (class 0), based on its shot angle. This way, a badminton video can be seen as a long sequence of high and other. Assuming that the scenes in the rallies are always shot from a high angle, we set the frame as the rally's start frame when the shot angle varies from other to high and the frame as the rally's end frame when the output varies from high to other. As a result, the rally-wise video trimming module can generate rally-related frames. Shot Angle Recognition We adopted the shot angle convolutional neural network (SA-CNN) model, as shown in Figure 1

==== KW: shot angle ====

gle, we set the frame as the rally's start frame when the shot angle varies from other to high and the frame as the rally's end frame when the output varies from high to other. As a result, the rally-wise video trimming module can generate rally-related frames. Shot Angle Recognition We adopted the shot angle convolutional neural network (SA-CNN) model, as shown in Figure 2, to solve the binary classification problem. The dataset we built for the shot angle recognition task comprises 112660 frames collected from 6 unique men's singles matches publicly available on the BWF channel. We divided them

==== KW: shot angle ====

ies from high to other. As a result, the rally-wise video trimming module can generate rally-related frames. Shot Angle Recognition We adopted the shot angle convolutional neural network (SA-CNN) model, as shown in Figure 2, to solve the binary classification problem. The dataset we built for the shot angle recognition task comprises 112660 frames collected from 6 unique men's singles matches publicly available on the BWF channel. We divided them into training and testing datasets, with 62970 (4 matches) and 49690 (2 matches) frames, respectively. Additionally, badminton coaches have labeled each

==== KW: shot angle ====

SA-CNN. Such dataset comprising of two matches (49690 frames), and all of its frames are labeled with one of the two classes, high and other. The confusion matrix for the evaluation is displayed in Table 2. We can observe that SA-CNN has remarkable performance and great accuracy for identifying shot angles. Rally-Wise Video Trimming While the frames were inputted sequentially into SA-CNN, a continuous sequence of high and other will be outputted. In this study, we evaluated our rally-wise video trimming module with S2-Labeling, a shot-by-shot labeling dataset that consists of six badminton matches

==== KW: shot angle ====

per, we developed an automated information extraction model that captures rally-by-rally information from publicly available badminton videos, bridging the gap between videos and analyzable information. We trimmed the rally frame sequences from raw badminton match videos based on the interchange of shot angle, detected hit-frames, which possess abundant player information, with the shifts in shuttlecock flying direction, and achieved 81%, 96% accuracy, respectively. Our approach can further support AI-based applications like stroke classification, movement recognition, and tactical analysis. The badminton

==== KW: Shot Angle ====

rallies are always shot from a high angle, we set the frame as the rally's start frame when the shot angle varies from other to high and the frame as the rally's end frame when the output varies from high to other. As a result, the rally-wise video trimming module can generate rally-related frames. Shot Angle Recognition We adopted the shot angle convolutional neural network (SA-CNN) model, as shown in Figure 2, to solve the binary classification problem. The dataset we built for the shot angle recognition task comprises 112660 frames collected from 6 unique men's singles matches publicly available

==== KW: trimming ====

in hit-frames can subsequently be utilized to synthesize players' strokes and on-court movement, as well as for other downstream applications such as analyzing training tasks and competition strategy. The proposed approach in this study comprises several automated procedures like rally-wise video trimming, player and court keypoints detection, shuttlecock flying direction prediction, and hit-frame detection. In the study, we achieved 99% accuracy on shot angle recognition for video trimming, over 92% accuracy for applying player keypoints

sequences on shuttlecock flying direction prediction, and

==== KW: trimming ====

strategy. The proposed approach in this study comprises several automated procedures like rally-wise video trimming, player and court keypoints detection, shuttlecock flying direction prediction, and hit-frame detection. In the study, we achieved 99% accuracy on shot angle recognition for video trimming, over 92% accuracy for applying player keypoints sequences on shuttlecock flying direction prediction, and reported the evaluation results of rally-wise video trimming and hit-frame detection. Introduction Badminton, the fastest racquet sport in the world, is played by players of all ages and can be

==== KW: trimming ====

flying direction prediction, and hit-frame detection. In the study, we achieved 99% accuracy on shot angle recognition for video trimming, over 92% accuracy for applying player keypoints sequences on shuttlecock flying direction prediction, and reported the evaluation results of rally-wise video trimming and hit-frame detection. Introduction Badminton, the fastest racquet sport in the world, is played by players of all ages and can be played in singles or doubles configurations (Phomsoupha and Laffaye 2015; Ramasamy et al. 2021). Badminton is also a fast-paced and explosive sport that requires athl

==== KW: trimming ====

cedures. The hit-frame generating algorithm inputs a badminton frame sequence (video)  $V$  and produces a set of hit-frame indices denoted by  $Hit$ . Lines 1 to 3 define the shot angle constants  $O$  (representing other) and  $H$  (representing high). These constants are used to realize the rally-wise video trimming module (described in ). The variable previous shot angle is initialized to  $O$  to keep track of the shot angle of the previous frame. Two sets are also created:  $Kps$  (for storing detected player keypoints) and  $Hit$  (for storing detected hit-frame indices), both initially empty. From line 4, the alg

==== KW: trimming ====

utomated hit-frame detection tool to bridge the gap between raw badminton videos and analyzable data. (a) Shot from a high angle (b) Shot from other angles Figure 1: Frame shot from different angles. Shuttlecock Flying Direction Prediction This section elaborates on the proposed rally-wise video trimming, human and court keypoint detection, player detection, and shuttlecock flying direction prediction procedures. We adopted broadcast badminton videos from the public BWF YouTube channel as our data source of the dataset we utilized in the following paragraphs. Rally-wise Video Trimming To analyze

==== KW: trimming ====

data source of the dataset we utilized in the following paragraphs. Rally-wise Video Trimming To analyze the video of badminton matches with a 21-point scoring system simultaneously is challenging; we reduced the mission to a point-by-point (rally-by-rally) task and implemented a rally-wise video trimming module to trim all the rally-related frames from the original video. These frame segments can represent player behaviors in each rally. We noticed that the match videos were shot from a high angle when the players competed on the court and shot from other angles while replaying highlights or film

==== KW: trimming ====

of high and other. Assuming that the scenes in the rallies are always shot from a high angle, we set the frame as the rally's start frame when the shot angle varies from other to high and the frame as the rally's end frame when the output varies from high to other. As a result, the rally-wise video trimming module can generate rally-related frames. Shot Angle Recognition We adopted the shot angle convolutional neural network (SA-CNN) model, as shown in Figure 2, to solve the binary classification problem. The dataset we built for the shot angle recognition task comprises 112660 frames collected fr

==== KW: trimming ====

ed 90% every six epochs. During the training phase of SA-CNN  $N = 8$ ,  $C = 2$ ,  $L = 3$  and the loss function  $L$  that we used is shown in Eq.1.  $L = l(y, y) = \{l_1 \dots l_n\}^T$  (1)  $l_n = -\log \exp(y_n, y_n)$  (2)  $l(y, y) = \sum_{n=1}^N l_n$  (3) Court and Player Keypoints Detection After the rally-wise video trimming module generated the frame sequence of a rally, we adopted R-CNN models to detect human and court keypoint sequences. These keypoint 64 54 32 108 32 27 2 Flatten Max Pooling Convolutional Layer Batch Normalization ReLU Fully Connected Layer Figure 2: The SA-CNN architecture. sequences showcase the

==== KW: trimming ====

2: Confusion matrix of the SA-CNN. Accuracy Precision Recall F1-Score 0.9976 0.9903 1.0 0.9951  
3. Can hit-frames be precisely identified through flying direction changes? We will answer these questions in the following paragraphs, and the assessment and experiment of the SA-CNN, rally-wise video trimming module, transformer, and hit-frame detecting module are the main topics of this section. SA-CNN We adopted the testing dataset mentioned in to evaluate SA-CNN. Such dataset comprising of two matches (49690 frames), and all of its frames are labeled with one of the two classes, high and other. The

==== KW: trimming ====

e 2. We can observe that SA-CNN has remarkable performance and great accuracy for identifying shot angles. Rally-Wise Video Trimming While the frames were inputted sequentially into SA-CNN, a continuous sequence of high and other will be outputted. In this study, we evaluated our rally-wise video trimming module with S2-Labeling, a shot-by-shot labeling dataset that consists of six badminton matches (Huang et al. 2022); however, we could only find four available video sources of the dataset so those accessible videos will be used for evaluation. The S2-Labeling dataset provides the rally's start Ta

==== KW: trimming ====

videos will be used for evaluation. The S2-Labeling dataset provides the rally's start Table 3: Numbers of correctly trimmed, extra trimmed, miss trimmed videos and the actual rally counts. Correct Extra Missed Total Trimmed Actual 287 33 36 320 323 Table 4: Confusion matrix of the rally-wise video trimming module. Accuracy Precision Recall F1-Score 0.8062 0.8969 0.8885 0.8927 and end frame number, which we adopted in the evaluation stage. Table 3 shows the number of rallies trimmed and the actual rally count, while Table 4 shows the confusion matrix of the SA-CNN. From Tables 3 and 4, we acknowl-

==== KW: trimming ====

recision Recall F1-Score 0.8062 0.8969 0.8885 0.8927 and end frame number, which we adopted in the evaluation stage. Table 3 shows the number of rallies trimmed and the actual rally count, while Table 4 shows the confusion matrix of the SA-CNN. From Tables 3 and 4, we acknowledged that the video trimming module could not trim all rally-related videos. Such a limitation is due to the varying shooting forms of broadcast badminton videos. This restriction is inevitable as the broadcasting department aims to enhance the audience's viewing experience. However, most of the time, the rally-wise video

==== KW: trimming ====

module could not trim all rally-related videos. Such a limitation is due to the varying shooting forms of broadcast badminton videos. This restriction is inevitable as the broadcasting department aims to enhance the audience's viewing experience. However, most of the time, the rally-wise video trimming module can still trim rally-related frames from raw match videos and achieve high precision and recall value. Transformer With the transformer model, player keypoint sequences can be transformed into shuttlecock flying direction sequences comprising steady (token S), flying toward the bottom court

==== KW: rally-wise ====

data included in hit-frames can subsequently be utilized to synthesize players' strokes and on-court movement, as well as for other downstream applications such as analyzing training tasks and competition strategy. The proposed approach in this study comprises several automated procedures like rally-wise video trimming, player and court keypoints detection, shuttlecock flying direction prediction, and hit-frame detection. In the study, we achieved 99% accuracy on shot angle recognition for video trimming, over 92% accuracy for applying player keypoints sequences on shuttlecock flying direction

==== KW: rally-wise ====

on, shuttlecock flying direction prediction, and hit-frame detection. In the study, we achieved 99% accuracy on shot angle recognition for video trimming, over 92% accuracy for applying player keypoints sequences on shuttlecock flying direction prediction, and reported the evaluation results of rally-wise video trimming and hit-frame detection. Introduction Badminton, the fastest racquet sport in the world, is played by players of all ages and can be played in singles or doubles configurations (Phomsoupha and Laffaye 2015; Ramasamy et al. 2021). Badminton is also a fast-paced and explosive sport tha

==== KW: rally-wise ====

ss of all the procedures. The hit-frame generating algorithm inputs a badminton frame sequence



(video)  $V$  and produces a set of hit-frame indices denoted by  $Hit$ . Lines 1 to 3 define the shot angle constants  $O$  (representing other) and  $H$  (representing high). These constants are used to realize the rally-wise video trimming module (described in ). The variable previous shot angle is initialized to  $O$  to keep track of the shot angle of the previous frame. Two sets are also created:  $Kps$  (for storing detected player keypoints) and  $Hit$  (for storing detected hit-frame indices), both initially empty. From

==== KW: rally-wise ====

$h$  frames like those in lines 8 and 9. After the algorithm processes all frames in the input sequence, it returns the set  $Hit$ , which contains detected hit-frames indices of the input video. Contribution The main contributions of our article are to: 1. Utilize the interchange of shot angle to trim rally-wise frames from raw badminton video. 2. Proposed a novel transformer that predicts shuttlecock direction sequences based on player keypoint sequence. 3. Designed and developed the first-ever automated hit-frame detection tool to bridge the gap between raw badminton videos and analyzable data. (a) Sh

==== KW: rally-wise ====

the first-ever automated hit-frame detection tool to bridge the gap between raw badminton videos and analyzable data. (a) Shot from a high angle (b) Shot from other angles Figure 1: Frame shot from different angles. Shuttlecock Flying Direction Prediction This section elaborates on the proposed rally-wise video trimming, human and court keypoint detection, player detection, and shuttlecock flying direction prediction procedures. We adopted broadcast badminton videos from the public BWF YouTube channel as our data source of the dataset we utilized in the following paragraphs. Rally-wise Video Trimm

==== KW: rally-wise ====

e channel as our data source of the dataset we utilized in the following paragraphs. Rally-wise Video Trimming To analyze the video of badminton matches with a 21-point scoring system simultaneously is challenging; we reduced the mission to a point-by-point (rally-by-rally) task and implemented a rally-wise video trimming module to trim all the rally-related frames from the original video. These frame segments can represent player behaviors in each rally. We noticed that the match videos were shot from a high angle when the players competed on the court and shot from other angles while replaying highl

## 11. assistant (2026-06-06T17:38:29.678Z)

I now have the full picture from the primary source. Let me analyze the claim carefully against what the paper actually says.

The claim has several components to check:

1. "deep-learning pipeline that includes rally-wise video trimming (a form of temporal rally segmentation)" ? TRUE. The paper explicitly describes "rally-wise video trimming" as trimming all rally-related frames from raw video, with rally start = shot angle transition "other ? high" and rally end = "high ? other." This IS temporal rally segmentation.
2. "using shot-angle recognition" ? TRUE. The trimming module is realized via the SA-CNN (Shot Angle CNN) binary classifier (high vs. other camera angle).
3. "achieving 99% accuracy on shot angle recognition for that trimming step" ? supported by the abstract verbatim, and Table 2 confirms SA-CNN accuracy = 0.9976 (?99%).

Now the critical scrutiny point: what does "shot angle" mean here, and is the 99% attributable to the trimming?

Key finding: "shot angle" here means CAMERA shot angle (broadcast camera framing: "high angle" when players are competing on court vs. "other angles" during replays/highlights), NOT badminton stroke/shot angle. The claim does not assert it's stroke angle, so this is consistent.

Important nuance: The 99% accuracy is for the SA-CNN frame-level shot-angle classification (Table 2: Accuracy 0.9976). The \*rally-wise video trimming module itself\* achieves a SEPARATELY reported, LOWER accuracy: 0.8062 (Table 4), with the conclusion reporting 81% for trimming. So the 99% belongs to the per-frame shot-angle recognition step (the component the trimming is built on), and the trimming step end-to-end is 81%.

The claim says "99% accuracy on shot angle recognition for that trimming step" ? this matches the abstract's exact phrasing ("99% accuracy on shot angle recognition for video trimming") and Table 2. The claim correctly scopes the 99% to "shot angle recognition" (not to the trimming

end-to-end). So the claim is accurately worded and not an overreach.

Let me verify the date and that this is a legitimate peer-reviewed/arXiv source, and confirm the table values once more.

## 12. user (2026-06-06T17:38:36.766Z)

== arXiv == minton matches by considering long-term and short-term dependencies. (Wang 2022) also introduced a framework with two arXiv:2307.16000v2 [cs.CV] 2 Aug 2023 encoder-decoder extractors and a position-aware fusion network to forecast the possible  
== AAAI == t: Position-aware fusion of rally progress and player styles for stroke forecasting in badminton. In Proceedings of the AAAI Conference on Artificial Intelligence, volume 36, 4219-4227. Wang, Z.; Guo, M.; and Zhao, C. 2016. Badminton stroke recognition: Predicting the players' striking location, the shuttlecock's landing zone, and their movements as they return the shuttlecock. Copyright © 2024, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved. This information can be used for further tactical analysis

## 13. assistant (2026-06-06T17:38:46.411Z)

All verified against the primary source. I have everything needed to rule on this claim.

Summary of verification against the primary source (arXiv:2307.16000v2, "Automated Hit-frame Detection for Badminton Match Analysis," Chien & Yu, 2023; AAAI copyright 2024):

- Component 1 ? "deep-learning pipeline that includes rally-wise video trimming (a form of temporal rally segmentation)": SUPPORTED. The paper explicitly implements a "rally-wise video trimming module" that sets rally start = shot-angle transition other?high and rally end = high?other, producing rally-related frame segments. This is temporal rally segmentation.
- Component 2 ? "using shot-angle recognition": SUPPORTED. Trimming is realized by the SA-CNN binary classifier ("high" vs "other" camera angle). Note: "shot angle" = broadcast CAMERA angle, not badminton stroke angle. The claim does not misrepresent this.
- Component 3 ? "99% accuracy on shot angle recognition for that trimming step": SUPPORTED. Abstract states verbatim "we achieved 99% accuracy on shot angle recognition for video trimming." Table 2 (SA-CNN confusion matrix): Accuracy 0.9976, Precision 0.9903, Recall 1.0, F1 0.9951 ? i.e., ~99%.

Important nuance the claim handles CORRECTLY: the 99% is for the per-frame shot-angle recognition component, NOT for the end-to-end trimming, which the paper separately reports at Accuracy 0.8062 / F1 0.8927 (Table 4) and as "81%" in the conclusion. The claim scopes 99% specifically to "shot angle recognition" ? matching the paper's own framing ? so it is not an overreach.

The claim is accurate, current for its 2023/2024 source, sourced to the primary arXiv paper, and properly scoped. No contradicting evidence; the claim is essentially a faithful paraphrase of the abstract plus Table 2.

## 14. user (2026-06-06T17:38:54.320Z)

Structured output provided successfully

## 15. assistant (2026-06-06T17:38:57.072Z)

I've completed the verification and submitted my structured output. The claim is NOT refuted (refuted=false, high confidence) ? it is a faithful, correctly-scoped paraphrase of the primary source's abstract and Table 2.